

Sistemas Distribuidos

Clase 03 — Consistencia, replicación y modelos de estado

Departamento de Computación

Ingeniería Informática – Comisión A – Pilar

Equipo docente – 11 de agosto de 2026

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Del RPC al estado

Sistemas Distribuidos

De la comunicación al estado

Clase 2:

- RPC bajo incertidumbre
- timeouts, retries, idempotencia

De la comunicación al estado

Clase 2:

- RPC bajo incertidumbre
- timeouts, retries, idempotencia

Nuevo problema

¿Cómo mantenemos **estado** cuando hay múltiples copias?

Replicación

Replicación = múltiples nodos mantienen el mismo *estado lógico*.

Replicación

Replicación = múltiples nodos mantienen el mismo *estado lógico*.
Motivaciones:

- disponibilidad
- latencia (lecturas cercanas)
- tolerancia a fallas

Replicación

Replicación = múltiples nodos mantienen el mismo *estado lógico*.
Motivaciones:

- disponibilidad
- latencia (lecturas cercanas)
- tolerancia a fallas

Hecho estructural

Sin coordinación perfecta, las réplicas **divergen**.

Por qué divergen

- latencias variables
- pérdidas/reintentos
- escrituras concurrentes
- particiones

Por qué divergen

- latencias variables
- pérdidas/reintentos
- escrituras concurrentes
- particiones

Conclusión

La divergencia no es un bug: es una consecuencia del modelo.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Modelos de consistencia

Sistemas Distribuidos

¿Qué es consistencia?

Consistencia define:

qué valores pueden observar los clientes y en qué orden.

Linearizabilidad (consistencia fuerte)

- equivalente a una sola copia
- respeta orden de tiempo real

Linearizabilidad (consistencia fuerte)

- equivalente a una sola copia
- respeta orden de tiempo real

Intuición

Cada operación ocurre en un punto único entre invocación y respuesta.

Causalidad (happens-before)

Relación $a \rightarrow b$ si:

- mismo proceso y a ocurre antes que b
- a envía mensaje que causa b
- la relación se propaga por transitividad

Causalidad (happens-before)

Relación $a \rightarrow b$ si:

- mismo proceso y a ocurre antes que b
- a envía mensaje que causa b
- la relación se propaga por transitividad

Consistencia causal

Respeta dependencias causales, no impone orden total.

Tiempo físico no alcanza

Comparar timestamps de nodos distintos puede engañar:

- relojes locales derivan o se corrigen;
- la red reordena o demora mensajes;
- una hora menor no prueba causalidad.

Tiempo físico no alcanza

Comparar timestamps de nodos distintos puede engañar:

- relojes locales derivan o se corrigen;
- la red reordena o demora mensajes;
- una hora menor no prueba causalidad.

Pregunta correcta

¿Qué evento pudo influir sobre qué otro evento?

Relojes de Lamport

Cada proceso mantiene un contador lógico:

- incrementa antes de cada evento local;
- adjunta el contador al enviar mensajes;
- al recibir, usa $\max(\textit{local}, \textit{recibido}) + 1$.

Relojes de Lamport

Cada proceso mantiene un contador lógico:

- incrementa antes de cada evento local;
- adjunta el contador al enviar mensajes;
- al recibir, usa $\max(\mathit{local}, \mathit{recibido}) + 1$.

$$a \to b \implies L(a) < L(b)$$

Límite

$L(a) < L(b)$ no prueba que $a \to b$.

Eventual consistency

- sin nuevas escrituras, las réplicas convergen
- no hay garantías inmediatas de lectura

Trade-off

- fuerte: coordinación + latencia + menor disponibilidad
- débil: disponibilidad + anomalías temporales

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Arquitecturas de replicación

Leader-based

- un líder serializa escrituras
- followers replican (sync/async)

Multi-leader

- múltiples líderes
- escritura local rápida
- conflictos inevitables

Leaderless (Dynamo)

- N réplicas
- escribir a W, leer de R
- condición clásica: $R + W > N$

Quorums

- write quorum W
- read quorum R

Quorums

- write quorum W
- read quorum R

Propiedad

Si $R + W > N$, hay intersección de quorums.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Conflictos y convergencia

Conflictos

- escrituras concurrentes sobre mismo objeto
- orden global no definido

Resolución

- LWW (last-write-wins)
- merge semántico
- versionado (vector clocks)

Vector clocks

Permiten comparar versiones:

- si un vector domina a otro, hay descendencia causal;
- si no son comparables, hay concurrencia.

Vector clocks

Permiten comparar versiones:

- si un vector domina a otro, hay descendencia causal;
- si no son comparables, hay concurrencia.

$$(2, 1, 0) \parallel (1, 2, 0)$$

Uso

Detectar conflicto y forzar merge en vez de sobrescribir datos.

Anti-entropy

- sincronización periódica entre réplicas
- gossip / read-repair

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Modelo del curso

Aplicación de $D = (P, F, G, S, T)$

- P: estado replicado
- F: delay, pérdida, partición
- G: (linearizable / causal / eventual)
- S: leader, multi-leader, quorum
- T: latencia, disponibilidad, complejidad

Cierre

Idea central

Replicar estado obliga a elegir *qué inconsistencias aceptar* y *cuándo*.